PROJET UNIVERSITAIRE · DEVOPS

AgriMétéo

Présentation du projet

Ce qui existe déjà, comment le projet est organisé, et qui travaille sur quoi. Support de la réunion d'équipe.

Informations

Établissement	Dakar Institute of Technology
Cours	DevOps
Enseignant	Monsieur Madicke Diop
Année universitaire	Licence 3, 2026-2027
État du dépôt	2 novembre 2026
Document de référence	AgriMétéo, Cahier des charges, version 1.0
Membres de l'équipe	Fatoumata Yarie Camara Ouleymatou Sidy Kane Roland Bintou Gueye Oumar Mame Coumba Laye N’doye

Version 1.0 · novembre 2026

Sommaire

1. L'essentiel **3**
1.1 Le projet en bref 3
1.2 Où en est le projet 3
2. Le produit **3**
2.1 Trois notions à ne pas confondre 3
2.2 Le parcours de l'utilisateur 3
2.3 Le périmètre du MVP 4
3. L'architecture **4**
3.1 Vue d'ensemble 4
3.2 Pourquoi ces séparations 5
3.3 Le pipeline 5
4. Qui travaille sur quoi **5**
4.1 Répartition proposée 6
4.2 Tâches de chaque lot 6
5. Les fichiers du dépôt **7**
5.1 Racine du dépôt 7
5.2 .github/ : GitHub 8
5.3 backend/ : API FastAPI 8
5.4 ml/ : analyse et règles 9
5.5 frontend/ : React et Vite 9
5.6 e2e/ : tests de bout en bout 9
5.7 infra/ : déploiement 10
5.8 scripts/ : contrôles et migration 10
5.9 docs/ : documentation 10
6. Comment on travaille **11**
6.1 Le cycle d'une tâche 11
6.2 Quatre règles à retenir 12
7. Démarrer sur son poste **12**
7.1 Avec Docker 12
7.2 Sans Docker, pour développer 12
8. État réel du projet **13**
8.1 Ce qui est prouvé 13
8.2 Anomalies et risques 13
9. Planning et décisions **13**
9.1 Les 14 jours 14
9.2 À décider en réunion 14
9.3 À valider avec M. Diop 14

1. L'essentiel

Le socle technique d'AgriMétéo fonctionne, mais l'application ne renvoie encore que des données factices : le travail produit des lots 1 à 4 reste à faire.

1.1 Le projet en bref

AgriMétéo est une application web qui estime si la météo récente est favorable à la mise en culture d'une parcelle, en donnant ses limites. C'est le projet DevOps de Licence 3 au Dakar Institute of Technology, pour M. Madicke Diop, réalisé en 14 jours par une équipe de six personnes.

1.2 Où en est le projet

- **Ce qui marche :** l'application démarre sous Docker, le déploiement et le retour arrière sont testés, et les lots 5 et 6 sont bien avancés.
- **Ce qui manque :** les vraies données météo, le moteur de règles et les écrans complets.
- **Pas encore sur GitHub :** le dépôt n'y est pas poussé, donc le pipeline n'y a jamais tourné.
- **Pour la réunion :** confirmer qui prend quel lot et trancher les décisions du chapitre 9.

2. Le produit

L'application ne dit jamais qu'un terrain est cultivable : elle évalue la météo, et seulement la météo. C'est la règle à garder en tête sur tous les lots.

2.1 Trois notions à ne pas confondre

Notion	Question posée	Dans le MVP
Favorabilité météo	La météo récente est-elle compatible avec la culture choisie ?	Calculée par un moteur de règles, sur plusieurs indicateurs
Humidité du sol	Quelle est l'humidité de la terre ?	Estimation de modèle, affichée comme indicative
Cultivabilité agronomique	Le terrain convient-il vraiment ?	Non évaluée : ni données de sol, ni observations de terrain

2.2 Le parcours de l'utilisateur

1. Il choisit une culture, un lieu, une date et une durée.
2. Il clique sur « Analyser ».
3. Il voit les trois notions, la qualité des données, l'indice de confiance et les limites.
4. Un tableau de bord montre les graphiques et les indicateurs.

2.3 Le périmètre du MVP

Dans le MVP	Hors du MVP
Une source réelle (Open-Meteo) et des données de test	Comptes utilisateurs, historique des analyses
Une API REST décrite par un contrat OpenAPI	Carte interactive, capteurs
Conteneurs, pipeline, déploiement avec retour arrière	Conseils agronomiques détaillés, prévision de rendement
Une ou deux cultures	Machine learning

> **Deux règles à respecter**
>
> Les seuils agronomiques ne s'inventent pas : chacun est sourcé ou marqué « à valider » à l'écran. Les tests et la démonstration doivent marcher sans Internet.

3. L'architecture

Trois modules, un contrat d'API et une seule porte d'entrée : le port 8080 du frontend. Le backend et `ml` partent dans la même image Docker.

3.1 Vue d'ensemble

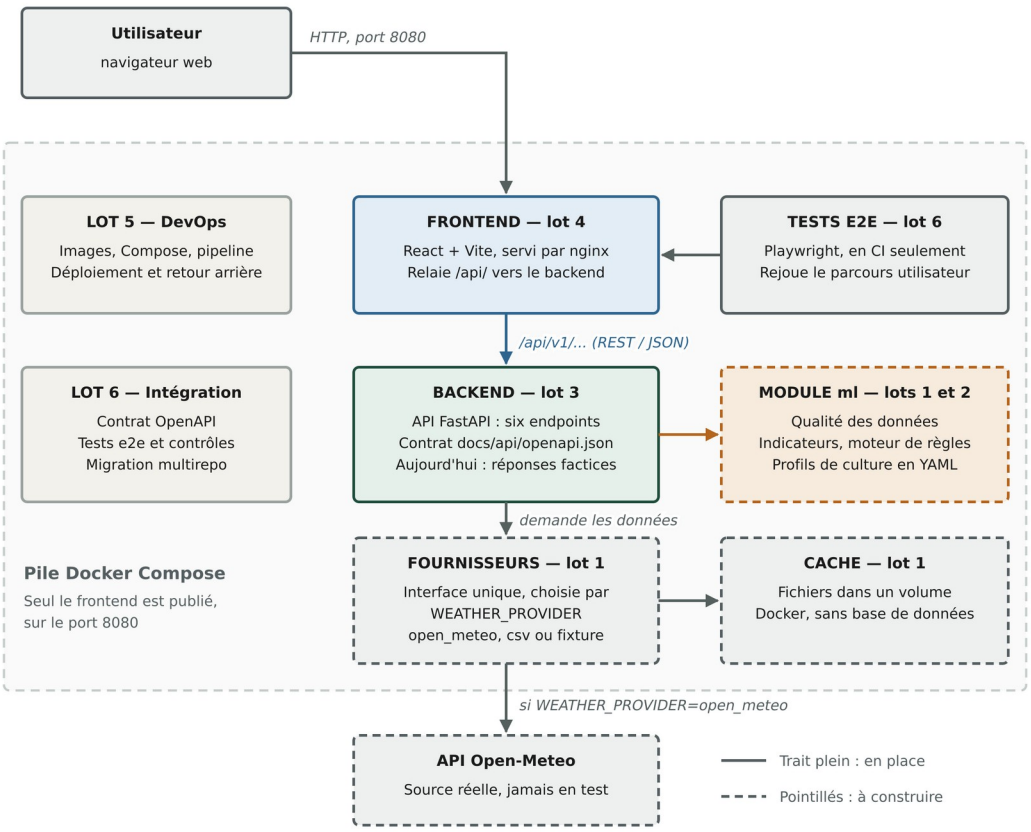


Figure 1 : Modules de l'application et lot responsable de chacun

Aujourd'hui, le backend répond avec `backend/app/services/stub.py` et n'appelle ni `ml` ni fournisseur : les lots 1 à 3 construisent ces liens. Le frontend affiche le résultat sans refaire aucun calcul.

3.2 Pourquoi ces séparations

Chaque lot avance en parallèle grâce au contrat `docs/api/openapi.json`. Le dépôt peut aussi être coupé en un dépôt frontend et un dépôt backend : l'essai de cette séparation a réussi.

Module	Peut utiliser	Ne doit jamais
frontend	L'API, via le contrat OpenAPI	Importer du code backend ou ml
backend	Le package ml et son propre code	Importer du code frontend
ml	Ses bibliothèques Python	Importer backend ou frontend
e2e et infra	Tous les modules, par les conteneurs	Être utilisés par les modules

Le pipeline vérifie ces règles à chaque exécution, avec `scripts/check_imports.py`.

3.3 Le pipeline

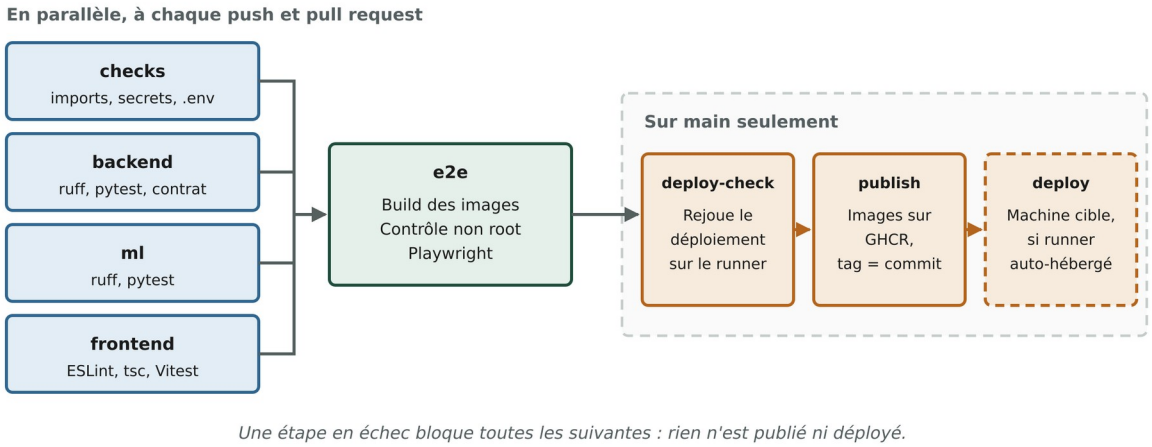


Figure 2 : Enchaînement des jobs GitHub Actions

Le cahier prévoit Jenkins, mais l'ADR-005 le remplace par GitHub Actions sur consigne de l'enseignant. Les six étapes du cahier sont conservées.

4. Qui travaille sur quoi

Proposition à valider en réunion

Le cahier des charges n'attribue aucun lot. Les noms ci-dessous suivent l'ordre de sa page de garde, sans tenir compte des compétences : échangez-les librement.

4.1 Répartition proposée

Lot	Responsable	Relecture	Mission	Dossiers
1. Données météo	Fatoumata Yarie Camara	Ouleymatou Sidy Kane	Fournir des données propres et documentées	`backend/app/providers/`, `ml/src/`, `ml/data/`
2. Moteur de décision	Ouleymatou Sidy Kane	Fatoumata Yarie Camara	Transformer les données en résultat expliqué	`ml/src/`, `ml/rules/`
3. Backend FastAPI	Roland	Bintou Gueye	Exposer le tout par une API fiable	`backend/`
4. Frontend web	Bintou Gueye	Roland	Fournir l'interface utilisateur	`frontend/`
5. DevOps et déploiement	Oumar	Mame Coumba Laye N'doye	Construire, tester et déployer automatiquement	`infra/`, `.github/workflows/`, `docker-compose.yml`
6. Architecture et intégration	Mame Coumba Laye N'doye	Oumar	Garder l'ensemble cohérent, préparer la migration	`e2e/`, `scripts/`, `docs/`

Les lots 5 et 6 ont moins de développement restant : leurs responsables peuvent aider les lots 1 à 4. Le lot 6 vérifie l'ensemble, mais ne fusionne pas seul les pull requests des autres lots.

4.2 Tâches de chaque lot

Lot 1 : données météo

Point de départ : `backend/app/providers/` est vide et `ml/src/agrimeteo_ml/schema.py` ne définit que les colonnes communes.

- Écrire le fournisseur Open-Meteo, le fournisseur CSV et le fournisseur de test (T1-01).
- Contrôler la qualité des données et gérer le cache.
- Fournir les CSV d'essai dans `ml/data/sample/`.
- Critères à faire passer : D-01 à D-06.

Lot 2 : moteur de décision

Point de départ : `ml/rules/culture_exemple_a.yaml` est un profil de démonstration, sans aucun seuil réel.

- Écrire les indicateurs et le moteur de règles (T2-01).
- Écrire les profils de culture, avec des seuils sourcés ou marqués « à valider ».
- Rédiger les explications affichées à l'utilisateur.
- Critères à faire passer : P-01 à P-07.

Lot 3 : backend

Point de départ : les six endpoints existent et répondent avec les données factices de `backend/app/services/stub.py`.

- Brancher les routes sur les fournisseurs et le moteur, sans changer le contrat (T3-01).
- Figer les versions des dépendances Python (T3-02).
- Compléter les tests d'API.

- Critères à faire passer : la série A.

Lot 4 : frontend

Point de départ : `frontend/src/App.tsx` est une seule page, avec des coordonnées fixes.

- Construire les écrans Analyse, Résultat et Tableau de bord, avec les états de chargement, de vide et d'erreur (T4-01).
- **Afficher les limites à l'écran**, avec `data-testid="limits"` (T4-02). C'est l'anomalie AN-01, et le cahier interdit d'abandonner ce point.
- Écrire les tests Vitest et MSW, puis traiter les alertes `npm audit` (T4-03).
- Critères à faire passer : la série FE.

Lot 5 : DevOps et déploiement

Point de départ : images, Compose, workflow, déploiement et retour arrière sont écrits. Sauf le workflow, ils ont aussi été exécutés en local.

- Créer le dépôt GitHub, pousser le code et obtenir un premier pipeline vert (T5-01).
- Prouver J-01 à J-05 sur GitHub, dont la démonstration avec un test cassé (T5-02).
- Protéger `main` et préparer la machine de démonstration (T5-03).

Lot 6 : architecture et intégration

Point de départ : contrat, client généré, contrôles automatiques, tests de bout en bout et essai de migration sont en place.

- Remplacer les noms provisoires de `.github/CODEOWNERS`, une fois les lots confirmés.
- Stabiliser le contrat et étendre les tests de bout en bout (T6-01).
- Remplacer la panne simulée du test I-03, une fois le lot 1 branché (T6-02).
- Refaire l'essai de migration sur le vrai dépôt (T6-03).

5. Les fichiers du dépôt

Chaque dossier a un lot responsable : avant de modifier un fichier, regardez la colonne Lot. Ne figurent pas ici les dossiers recréés automatiquement et ignorés par Git : `node_modules/`, `dist/`, `reports/`.

5.1 Racine du dépôt

Fichier	À quoi il sert	Lot
`README.md`	Présentation et démarrage rapide	6
`CONTRIBUTING.md`	Règles de travail : branches, commits, pull requests	6
`AgriMeteo_Cahier_des_charges.docx`	Le cahier des charges, référence du projet	Tous
`Makefile`	Raccourcis : `make up`, `make ci`, `make deploy`, `make rollback`	5
`docker-compose.yml`	Définit la pile : frontend publié, backend interne	5
`.env.example`	Modèle des variables, sans aucun secret	5
`.env`	Vos valeurs locales, jamais committées	Chacun
`.dockerignore`	Ce qui n'entre pas dans les images Docker	5

Fichier	À quoi il sert	Lot
`.gitignore`	Ce que Git ignore	6
`.gitattributes`	Impose les fins de ligne LF	6
`.editorconfig`	Style commun dans les éditeurs	6

5.2 .github/ : GitHub

Fichier	À quoi il sert	Lot
`workflows/ci.yml`	Le pipeline : tests, e2e, publication, déploiement	5
`CODEOWNERS`	Relecteurs par dossier, noms encore provisoires	5, 6
`pull_request_template.md`	Modèle de pull request	6
`ISSUE_TEMPLATE/tache.md`	Modèle d'issue pour une tâche	6
`ISSUE_TEMPLATE/anomalie.md`	Modèle d'issue pour une anomalie	6

5.3 backend/ : API FastAPI

Fichier	À quoi il sert	Lot
`app/main.py`	Point d'entrée : application, CORS, erreurs communes, `/health`	3
`app/api/v1/routes.py`	Les routes `/api/v1/...`	3
`app/core/config.py`	Lecture des variables d'environnement	3
`app/core/errors.py`	Format d'erreur commun	3
`app/schemas/common.py`	Schémas partagés : santé, erreur, position	3
`app/schemas/weather.py`	Schémas du géocodage et de la météo	3
`app/schemas/indicators.py`	Schémas des indicateurs	3
`app/schemas/predictions.py`	Schémas des cultures et de l'analyse	3
`app/services/stub.py`	**Données factices, à remplacer**	3
`app/providers/__init__.py`	Dossier vide qui recevra les fournisseurs météo	1
`app/export_openapi.py`	Génère le contrat OpenAPI	3
Les six autres `__init__.py`	Marquent les paquets Python ; celui de `app/` porte la version	3
`tests/test_api.py`	Tests d'API (9)	3
`tests/conftest.py`	Prépare le client de test	3
`Dockerfile`	Construit l'image, en utilisateur non root	5
`Makefile`	`make setup`, `lint`, `test`, `openapi`, `ci`	3
`pyproject.toml`	Dépendances et réglages Python	3
`README.md`	Mode d'emploi du module	3

5.4 ml/ : analyse et règles

Fichier	À quoi il sert	Lot
`src/agrimeteo_ml/schema.py`	Colonnes communes des données météo	1, 2
`src/agrimeteo_ml/__init__.py`	Marque le paquet Python	2
`rules/culture_exemple_a.yaml`	Profil de culture de démonstration, sans seuils réels	2
`data/sample/README.md`	Consignes pour les CSV d'essai à fournir	1
`tests/test_schema.py`	Test du schéma (1)	1, 2
`Makefile`	`make setup`, `lint`, `test`, `ci`	2
`pyproject.toml`	Dépendances et réglages Python	2

5.5 frontend/ : React et Vite

Fichier	À quoi il sert	Lot
`src/App.tsx`	L'unique page actuelle	4
`src/main.tsx`	Monte l'application React	4
`src/api.ts`	Appels à l'API	4
`src/api-types.gen.ts`	Types générés depuis le contrat, **à ne pas modifier à la main**	6
`index.html`	Page HTML d'accueil	4
`tests/App.test.tsx`	Tests de composants (2)	4
`tests/setup.ts`	Prépare les tests	4
`Dockerfile`	Construit l'image en deux temps : build, puis nginx non root	5
`nginx.conf`	Sert le site et relaie `/api/` vers le backend	5
`.dockerignore`, `.gitignore`	Exclusions pour Docker et Git	5
`Makefile`	`make setup`, `lint`, `test`, `build`, `ci`, `gen-api`, `check-client`	4
`package.json`, `package-lock.json`	Dépendances et versions verrouillées	4
`tsconfig.json`	Réglages TypeScript	4
`vite.config.ts`	Réglages de Vite et de Vitest	4
`eslint.config.js`	Règles du lint	4
`README.md`	Mode d'emploi du module	4

5.6 e2e/ : tests de bout en bout

Fichier	À quoi il sert	Lot
`tests/analyse.spec.ts`	Parcours dans le navigateur : analyse, panne, limites (en attente)	6
`tests/api.spec.ts`	API à travers nginx : smoke test et format d'erreur	6
`playwright.config.ts`	Réglages de Playwright	6

Fichier	À quoi il sert	Lot
`Dockerfile`, `.dockerignore`	Image qui lance les tests	6
`Makefile`, `README.md`	Commande de test et mode d'emploi	6
`package.json`, `package-lock.json`	Dépendances et versions verrouillées	6

5.7 infra/ : déploiement

Fichier	À quoi il sert	Lot
`scripts/deploy-local.sh`	Déploie, vérifie, revient en arrière seul en cas d'échec	5
`scripts/rollback.sh`	Relance la version précédente	5
`scripts/smoke-test.sh`	Vérifie le site, l'API et une analyse	5
`scripts/verify-deploy.sh`	Rejoue tout le scénario sur une pile jetable	5
`scripts/check-images.sh`	Contrôle : images non root et sans secret	5
`scripts/run-e2e-ci.sh`	Lance les tests de bout en bout en conteneur	5
`compose/docker-compose.dev.yml`	Développement : expose aussi le backend sur le port 8000	5
`compose/docker-compose.ci.yml`	CI : aucun port publié, plus le conteneur e2e	5
`deploy/.gitkeep`	Garde le dossier dans Git	5
`deploy/*.tag`, `deploy/history`	Versions déployées sur cette machine, ignorées par Git	5

5.8 scripts/ : contrôles et migration

Fichier	À quoi il sert	Lot
`check_imports.py`	Refuse les dépendances interdites entre modules	6
`check_env_example.py`	Refuse une variable absente de `.env.example`	5
`check_openapi.py`	Refuse un contrat différent de l'API réelle	6
`check_rules_profiles.py`	Vérifie les profils de culture	6
`check_secrets.py`	Refuse les secrets dans le dépôt	5
`create_labels.sh`, `create_issues.sh`	Créent les étiquettes et les issues GitHub depuis le backlog	6
`migration/extraire-depots.sh`	Extrait les dépôts frontend et backend	6
`migration/verifier-depots.sh`	Vérifie chaque dépôt extrait	6
`migration/backend.Makefile`	Devient le Makefile racine du dépôt backend	6

5.9 docs/ : documentation

Fichier	À quoi il sert	Lot
`guide-lots-5-6.md`	Guide détaillé du pipeline, du déploiement et des contrôles	5, 6
`exploitation.md`	Déployer, vérifier, revenir en arrière	5
`migration-multirepo.md`	Procédure de séparation des dépôts	6

Fichier	À quoi il sert	Lot
`api/openapi.json`	**Le contrat d'API**, généré depuis le code	3, 6
`architecture/overview.md`	Vue d'ensemble de l'architecture	6
`decisions/ADR-000-template.md`	Modèle pour une nouvelle décision	6
`decisions/ADR-001-monorepo.md`	Un dépôt unique, prêt à être séparé	6
`decisions/ADR-002-ci-sans-ports.md`	CI sans port publié	5
`decisions/ADR-003-regles-vs-ml.md`	Règles ou ML : **décision ouverte**	2
`decisions/ADR-004-fournisseur-meteo.md`	Open-Meteo proposé, **à confirmer**	1
`decisions/ADR-005-github-actions.md`	GitHub Actions à la place de Jenkins	5
`suivi/backlog.csv`	Liste des tâches	6
`suivi/anomalies.md`	Anomalies ouvertes et corrigées	Tous
`suivi/registre-risques.md`	Risques suivis	6
`suivi/definition-of-ready-done.md`	Quand une tâche est prête, et quand elle est finie	6
`suivi/projet-github.md`	Mise en place du suivi sur GitHub	6
`suivi/rapport-essai-migration.md`	Résultat de l'essai de migration	6
`suivi/rapport-validation-modele.md`	Sans objet tant qu'il n'y a pas de modèle ML	2
`preuves/README.md`	Ce qu'est une preuve dans ce projet	6
`preuves/J01/verifications-initiales.md`	Vérifications du premier jour	6
`preuves/J01/lots-5-6/README.md`	Index des sept journaux d'exécution, avec leur environnement	5, 6
`preuves/J01/lots-5-6/01` à `07` `*.log`	Journaux : contrôles, frontend, déploiement, e2e, migration, contrôles négatifs, déploiement réel	5, 6

6. Comment on travaille

Personne ne pousse directement sur `main` : tout passe par une branche courte, une pull request, une relecture et un pipeline vert.

6.1 Le cycle d'une tâche

1. Prendre une issue, par exemple T1-03, et la passer en « En cours ».
2. Créer la branche `feature/T1-03-slug`.
3. Faire de petits commits au format `feat(lot1): résumé [T1-03]`.
4. Ouvrir une pull request titrée `[T1-03] ...`, avec `Closes #n` dans son texte.
5. Obtenir au moins une relecture et un pipeline vert.
6. Fusionner, puis passer l'issue en « Validé ».

Statuts d'une tâche : Backlog, Prêt, En cours, En revue, Validé ou Bloqué.

Une tâche est finie quand : le code est relu, les tests et le lint passent en CI, une preuve est jointe dans `docs/preuves/`, la documentation est à jour et le statut a changé.

6.2 Quatre règles à retenir

- **Contrat d'abord.** Le frontend et le backend ne se parlent que par l'API. Si elle change : `make -C backend openapi`, `make -C frontend gen-api`, commit des deux fichiers et mention du lot 6 dans la pull request.
- **Modules étanches.** `ml` n'importe ni `backend` ni `frontend`, et le `frontend` n'importe ni l'un ni l'autre. Le pipeline le vérifie.
- **Honnêteté.** Un test qui échoue n'est pas « validé », et une preuve est une sortie réelle de commande.
- **Aucun secret dans Git.** Le fichier `.env` reste sur votre poste.

7. Démarrer sur son poste

Deux commandes suffisent avec Docker, depuis la racine du projet. Il faut Docker Desktop lancé et Compose 2.24 ou plus récent ; sous Windows, utilisez Git Bash.

7.1 Avec Docker

```
cp .env.example .env
sh infra/scripts/deploy-local.sh
```

Ouvrez ensuite http://localhost:8080. Pour arrêter : `docker compose down`.

Le dépôt n'étant pas encore sur GitHub, le code se récupère pour l'instant auprès de la personne qui l'a préparé.

7.2 Sans Docker, pour développer

Utilisez Python 3.12 et Node 22, les versions de la CI.

```
# API sur http://localhost:8000/docs
python -m venv .venv && . .venv/Scripts/activate   # Linux ou Mac : .venv/bin/activate
pip install -e "ml[dev]" -e "backend[dev]"
uvicorn app.main:app --app-dir backend --reload

# Interface sur http://localhost:5173, dans un second terminal
cd frontend && npm ci && npm run dev
```

Cette installation sans Docker n'a pas été testée sous Windows. Les commandes `make` non plus : `make` n'y est pas installé d'office (`winget install ezwinports.make`).

> **Dossier frontend en rouge dans VS Code ?**
> Lancez `npm ci` dans `frontend/` : les types viennent de `node_modules`, absent après une récupération du code.

8. État réel du projet

La chaîne technique est prouvée sur un poste Windows le 2 novembre 2026, mais rien n'a tourné sur GitHub et aucune donnée affichée n'est réelle.

8.1 Ce qui est prouvé

Sujet	État	Précision
Images Docker et démarrage	Prouvé	Services « healthy », utilisateurs non root
Déploiement et retour arrière	Prouvé	Retour automatique en cas d'échec, et retour manuel
Tests backend, ml, frontend	Prouvé	9, 1 et 2 tests réussis, mais ils portent sur un squelette
Tests de bout en bout	Prouvé	4 réussis, 1 en attente du lot 4
Contrôles automatiques	Prouvé	Contrôles Python testés avec des violations volontaires
Essai de migration	Prouvé	Sur un historique de test, à refaire sur le vrai dépôt
Panne du fournisseur	Partiel	Panne simulée dans le navigateur
Pipeline GitHub Actions (J-01 à J-05)	Non prouvé	Fichier validé par `actionlint`, jamais exécuté
Vraies données météo (lot 1)	Non commencé	Réponses factices
Moteur de règles et seuils (lot 2)	Non commencé	Profil de démonstration seulement
Écrans complets (lot 4)	Non commencé	Une page, coordonnées fixes, limites non affichées

8.2 Anomalies et risques

Anomalie	Description	Gravité
AN-01	Les limites renvoyées par l'API ne sont pas affichées	Haute
AN-05	11 alertes `npm audit` modérées, seulement dans les outils de développement	Faible

- **Pipeline jamais exécuté :** un premier lancement peut révéler d'autres problèmes.
- **Seuils agronomiques sans source :** sans références, les résultats resteront « à valider ».
- **Exigence de ML inconnue :** à clarifier avec M. Diop avant le jalon 2.
- **Machine de démonstration non choisie.**
- **Versions Python non figées :** une nouvelle version peut changer le contrat et faire échouer le pipeline.

Le détail est dans `docs/suivi/registre-risques.md` et `docs/guide-lots-5-6.md`.

9. Planning et décisions

Le projet dure 14 jours consécutifs, week-ends compris, et le périmètre est gelé après le jour 2. La date du jour 1 et celle du rendu restent à fixer : le cahier indique encore « jj/mm/aaaa ».

9.1 Les 14 jours

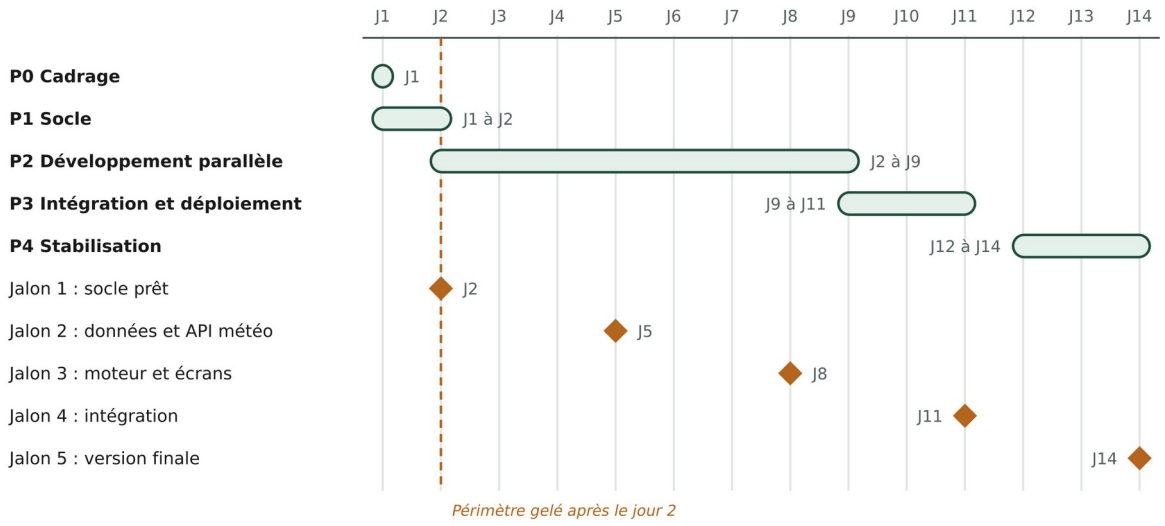


Figure 3 : Phases et jalons sur 14 jours

Les lots dépendent les uns des autres : données au jour 3, moteur au jour 8, scénarios complets au jour 12. Un lot en retard le signale tout de suite, et les autres continuent avec la version simulée.

> **Si le retard s'installe**
>
> On abandonne dans cet ordre : l'essai de migration (réduit à une extraction), puis une partie des graphiques, puis les finitions d'accessibilité. On n'abandonne jamais le contrat d'API, la CI, le déploiement avec retour arrière, le scénario nominal et l'affichage des limites.

9.2 À décider en réunion

- Confirmer le responsable et le relecteur de chaque lot.
- Fixer le jour 1, et l'heure du point quotidien de 15 minutes.
- Désigner qui crée le dépôt GitHub et y pousse le code (lot 5 proposé).
- Confirmer Open-Meteo comme fournisseur (ADR-004).
- Trancher la carte : le backlog la mentionne dans T4-01, alors que le cahier l'exclut du MVP.

9.3 À valider avec M. Diop

- Les cultures retenues : le cahier demande « au moins deux » à un endroit et « une ou deux » à un autre.
- La zone de démonstration, par exemple autour de Dakar.
- La machine de démonstration, et le fournisseur utilisé ce jour-là : données de test ou API réelle.
- La date et le format du rendu.
- Le niveau attendu sur la migration : deux dépôts par défaut, ou migration complète.
- Le machine learning, avant le jalon 2 (ADR-003).
- GitHub Actions à la place de Jenkins : l'ADR-005 cite sa consigne, le cahier reste à mettre à jour.